



# Object Oriented Programming Language (JAVA)

## Lecture 08

Course Code: CSE 06132133

Credit: 3.0

---

## Chapter 8: Inheritance



# Inheritance Basics

- **Inheritance** allows the creation of **hierarchical classifications**.
- In the terminology of Java, a **class** that is **inherited** is called a **superclass**. The class that does the **inheriting** is called a **subclass**.
- Therefore, a **subclass** is a specialized version of a **superclass**. It **inherits** all of the **members** defined by the **superclass** and adds its own, **unique** elements.
- To **inherit** a **class**, you simply incorporate the definition of one class into another by using the **extends** keyword.
- The general form of a class declaration that inherits a superclass is shown here:

```
class subclass-name extends superclass-name {  
    // body of class  
}
```



# Inheritance Basics

- Being a **superclass** for a **subclass** does not mean that the **superclass** cannot be used by itself. Further, a **subclass** can be a **superclass** for another **subclass**.
- You can only specify **one superclass** for any **subclass** that you create. Java **does not support** the inheritance of **multiple superclasses** into a **single subclass**.
- However, **no class** can be a **superclass** of **itself**.

```
1  // Create a superclass.
2  class A {
3      int i, j;
4      void showij() {
5          System.out.println("i and j: " + i + " " + j);
6      }
7  }
8  // Create a subclass by extending class A.
9  class B extends A {
10     int k;
11     void showk() {
12         System.out.println("k: " + k);
13     }
14     void sum() {
15         System.out.println("i+j+k: " + (i+j+k));
16     }
17 }
```

```
1 class SimpleInheritance {
2     public static void main(String[] args) {
3         A superOb = new A();
4         B subOb = new B();
5         // The superclass may be used by itself.
6         superOb.i = 10;
7         superOb.j = 20;
8         System.out.println("Contents of superOb: ");
9         superOb.showij();
10        System.out.println();
11        // The subclass has access to all public members of its superclass.
12        subOb.i = 7;
13        subOb.j = 8;
14        subOb.k = 9;
15        System.out.println("Contents of subOb: ");
16        subOb.showij();
17        subOb.showk();
18        System.out.println();
19        System.out.println("Sum of i, j and k in subOb:");
20        subOb.sum();
21    }
22 }
```

### Output:

Contents of superOb:

i and j: 10 20

Contents of subOb:

i and j: 7 8

k: 9

Sum of i, j and k in subOb:

i+j+k: 24



## Member Access and Inheritance

- Although a **subclass** includes all of the **members** of its **superclass**, it **cannot access** those **members** of the **superclass** that have been declared as **private**.



```
1  class A {
2      int i; // default access
3      private int j; // private to A
4      void setij(int x, int y) {
5          i = x;
6          j = y;
7      }
8  }
9  // A's j is not accessible here.
10 class B extends A {
11     int total;
12     void sum() {
13         total = i + j; // ERROR, j is not accessible here
14     }
15 }
16 class Access {
17     public static void main(String[] args) {
18         B subOb = new B();
19         subOb.setij(10, 12);
20         subOb.sum();
21         System.out.println("Total is " + subOb.total);
22     }
23 }
```



## A More Practical Example

```
1 class Box {
2     double width,height,depth;
3     Box(Box ob) { // pass object to constructor
4         width = ob.width; height = ob.height; depth = ob.depth;
5     }
6     Box(double w, double h, double d) { width = w; height = h; depth = d; }
7     Box() { width = -1; height = -1; depth = -1; }
8     Box(double len) { width = height = depth = len; }
9     double volume() { return width * height * depth; }
10 }
```



## A More Practical Example

```
1 class BoxWeight extends Box {  
2     double weight; // weight of box  
3     BoxWeight(double w, double h, double d, double m) {  
4         width = w;  
5         height = h;  
6         depth = d;  
7         weight = m;  
8     }  
9 }
```



## A More Practical Example

```
1 class DemoBoxWeight {
2     public static void main(String[] args) {
3         BoxWeight mybox1 = new BoxWeight(10, 20, 15, 34.3);
4         BoxWeight mybox2 = new BoxWeight(2, 3, 4, 0.076);
5         double vol;
6         vol = mybox1.volume();
7         System.out.println("Volume of mybox1 is " + vol);
8         System.out.println("Weight of mybox1 is " + mybox1.weight);
9         System.out.println();
10        vol = mybox2.volume();
11        System.out.println("Volume of mybox2 is " + vol);
12        System.out.println("Weight of mybox2 is " + mybox2.weight);
13    }
14 }
```

Output:

Volume of mybox1 is 3000.0

Weight of mybox1 is 34.3

Volume of mybox2 is 24.0

Weight of mybox2 is 0.076



## A More Practical Example

```
1 // Here, Box is extended to include color.
2 class ColorBox extends Box {
3     int color; // color of box
4     ColorBox(double w, double h, double d, int c) {
5         width = w;
6         height = h;
7         depth = d;
8         color = c;
9     }
10 }
```



## A Superclass Variable Can Reference a Subclass Object

- A **reference** variable of a **superclass** can be assigned a **reference** to any **subclass** derived from that **superclass**. You will find this aspect of **inheritance** quite useful in a variety of situations.

```
1  class RefDemo {
2      public static void main(String[] args) {
3          BoxWeight weightbox = new BoxWeight(3, 5, 7, 8.37);
4          Box plainbox = new Box();
5          double vol;
6          vol = weightbox.volume();
7          System.out.println("Volume of weightbox is " + vol);
8          System.out.println("Weight of weightbox is " + weightbox.weight);
9          System.out.println();
10         // assign BoxWeight reference to Box reference
11         plainbox = weightbox;
12         vol = plainbox.volume(); // OK, volume() defined in Box
13         System.out.println("Volume of plainbox is " + vol);
14         /* The following statement is invalid because plainbox
15         does not define a weight member. */
16         // System.out.println("Weight of plainbox is " + plainbox.weight);
17     }
18 }
```



## Using super

- Whenever a **subclass** needs to refer to its **immediate superclass**, it can do so by use of the keyword **super**.
- **super** has **two** general forms.
- The **first** calls the **superclass' constructor**.
- The **second** is used to **access a member** of the **superclass** that has been **hidden** by a **member** of a **subclass**.



## Using super to Call Superclass Constructors

- A **subclass** can call a **constructor** defined by its **superclass** by use of the following form of super:  
**super(*arg-list*);**
- Here, **arg-list** specifies any **arguments** needed by the **constructor** in the **superclass**.
- **super( )** must always be the **first statement** executed inside a **subclass' constructor**.



## Using super to Call Superclass Constructors

```
1 // BoxWeight now uses super to initialize its Box attributes.
2 class BoxWeight extends Box {
3     double weight; // weight of box
4     // initialize width, height, and depth using super()
5     BoxWeight(double w, double h, double d, double m) {
6         super(w, h, d); // call superclass constructor
7         weight = m;
8     }
9 }
```



## A Second Use for super

- The **second form** of **super** acts somewhat like this, except that it always refers to the **superclass** of the **subclass** in which it is used.
- This usage has the following general form:  
**super . *member***
- Here, **member** can be either a method or an **instance variable**.
- This second form of **super** is most applicable to situations in which **member** names of a **subclass** **hide** members by the **same name** in the **superclass**.

```
1  class A {
2      int i;
3  }
4  class B extends A {
5      int i; // this i hides the i in A
6      B(int a, int b) {
7          super.i = a; // i in A
8          i = b; // i in B
9      }
10     void show() {
11         System.out.println("i in superclass: " + super.i);
12         System.out.println("i in subclass: " + i);
13     }
14 }
15 class UseSuper {
16     public static void main(String[] args) {
17         B subOb = new B(1, 2);
18         subOb.show();
19     }
20 }
```

#### Output:

i in superclass: 1  
i in subclass: 2



## Creating a Multilevel Hierarchy

```
1 class Box {
2     private double width;
3     private double height;
4     private double depth;
5     Box(Box ob) { // pass object to constructor
6         width = ob.width; height = ob.height; depth = ob.depth;
7     }
8     Box(double w, double h, double d) { width = w; height = h; depth = d; }
9     Box() { width = -1; height = -1; depth = -1; }
10    Box(double len) { width = height = depth = len; }
11    double volume() { return width * height * depth; }
12 }
```



## Creating a Multilevel Hierarchy

```
1 class BoxWeight extends Box {  
2     double weight; // weight of box  
3     BoxWeight(BoxWeight ob) { super(ob); weight = ob.weight; }  
4     BoxWeight(double w, double h, double d, double m) {  
5         super(w, h, d); weight = m; }  
6     BoxWeight() { super(); weight = -1; }  
7     BoxWeight(double len, double m) { super(len); weight = m; }  
8 }
```



## Creating a Multilevel Hierarchy

```
1 class Shipment extends BoxWeight {
2     double cost;
3     Shipment(Shipment ob) { super(ob); cost = ob.cost; }
4     Shipment(double w, double h, double d, double m, double c) {
5         super(w, h, d, m); cost = c; }
6     Shipment() { super(); cost = -1; }
7     Shipment(double len, double m, double c) {
8         super(len, m); cost = c; }
9 }
```

```
1  class DemoShipment {
2      public static void main(String[] args) {
3          Shipment shipment1 = new Shipment(10, 20, 15, 10, 3.41);
4          Shipment shipment2 = new Shipment(2, 3, 4, 0.76, 1.28);
5          double vol;
6          vol = shipment1.volume();
7          System.out.println("Volume of shipment1 is " + vol);
8          System.out.println("Weight of shipment1 is " + shipment1.weight);
9          System.out.println("Shipping cost: $" + shipment1.cost);
10         System.out.println();
11         vol = shipment2.volume();
12         System.out.println("Volume of shipment2 is " + vol);
13         System.out.println("Weight of shipment2 is " + shipment2.weight);
14         System.out.println("Shipping cost: $" + shipment2.cost);
15     }
16 }
```



## When Constructors Are Executed

- The answer is that in a **class hierarchy**, **constructors** complete their **execution** in **order of derivation**, from **superclass** to **subclass**.
- Further, since **super( )** must be the **first** statement executed in a **subclass' constructor**, this order is the same whether or not **super( )** is used.
- If **super( )** is **not used**, then the **default** or parameterless **constructor** of each **superclass** will be executed.

```
1  class A {
2      A() {
3          System.out.println("Inside A's constructor.");
4      }
5  }
6  class B extends A {
7      B() {
8          System.out.println("Inside B's constructor.");
9      }
10 }
11 class C extends B {
12     C() {
13         System.out.println("Inside C's constructor.");
14     }
15 }
16 class CallingCons {
17     public static void main(String[] args) {
18         C c = new C();
19     }
20 }
```

### Output:

Inside A's constructor  
Inside B's constructor  
Inside C's constructor



# Method Overriding

- In a **class hierarchy**, when a **method** in a **subclass** has the **same name** and **type signature** as a **method** in its **superclass**, then the **method** in the **subclass** is said to **override** the **method** in the **superclass**.
- When an **overridden method** is called through its **subclass**, it will always **refer** to the version of that method defined by the **subclass**.
- The **version** of the **method** defined by the **superclass** will be **hidden**.

```
1 class A {
2     int i, j;
3     A(int a, int b) { i = a; j = b; }
4     void show() { System.out.println("i and j: " + i + " " + j); }
5 }
6 class B extends A {
7     int k;
8     B(int a, int b, int c) { super(a, b); k = c; }
9     void show() { System.out.println("k: " + k); }
10 }
11 class Override {
12     public static void main(String[] args) {
13         B subOb = new B(1, 2, 3);
14         subOb.show(); // this calls show() in B
15     }
16 }
```

```
1  class B extends A {  
2      int k;  
3      B(int a, int b, int c) { super(a, b); k = c; }  
4      void show() {  
5          super.show(); // this calls A's show()  
6          System.out.println("k: " + k);  
7      }  
8  }
```

Output:

i and j: 1 2

k: 3

```
1 class A {
2     int i, j;
3     A(int a, int b) { i = a; j = b; }
4     void show() { System.out.println("i and j: " + i + " " + j); }
5 }
6 class B extends A {
7     int k;
8     B(int a, int b, int c) { super(a, b); k = c; }
9     void show(String msg) { System.out.println(msg + k); }
10 }
11 class Override {
12     public static void main(String[] args) {
13         B subOb = new B(1, 2, 3);
14         subOb.show("This is k: "); // this calls show() in B
15         subOb.show(); // this calls show() in A
16     }
17 }
```



# Dynamic Method Dispatch

- **Method overriding** forms the basis for one of **Java's most powerful concepts: dynamic method dispatch**.
- **Dynamic method dispatch** is the mechanism by which a call to an **overridden method** is **resolved at run time**, rather than compile time.
- **Dynamic method dispatch** is important because this is how Java implements **run-time polymorphism**.
- When an **overridden method** is called through a **superclass reference**, Java determines which version of that method to execute based upon the type of the object being referred to at the time the call occurs. Thus, this determination is made at **run time**.
- When different types of objects are referred to, different versions of an overridden method will be called.
- In other words, ***it is the type of the object being referred to*** (not the type of the reference variable) that determines which version of an overridden method will be executed.
- Therefore, if a **superclass** contains a method that is **overridden** by a **subclass**, then when different types of objects are referred to through a **superclass reference variable**, different versions of the method are executed.

```
1 class A { void callme() { System.out.println("Inside A's callme method"); } }
2 class B extends A { void callme() { System.out.println("Inside B's callme method"); } }
3 class C extends A { void callme() { System.out.println("Inside C's callme method"); } }
4 class Dispatch {
5     public static void main(String[] args) {
6         A a = new A(); // object of type A
7         B b = new B(); // object of type B
8         C c = new C(); // object of type C
9         A r; // obtain a reference of type A
10        r = a; // r refers to an A object
11        r.callme(); // calls A's version of callme
12        r = b; // r refers to a B object
13        r.callme(); // calls B's version of callme
14        r = c; // r refers to a C object
15        r.callme(); // calls C's version of callme
16    }
17 }
```

```
1 class A { void callme() { System.out.println("Inside A's callme method"); } }
2 class B extends A { void callme() { System.out.println("Inside B's callme method"); } }
3 class C extends A { void callme() { System.out.println("Inside C's callme method"); } }
4 class Dispatch {
5     public static void main(String[] args) {
6         A a = new A(); // object of type A
7         B b = new B(); // object of type B
8         C c = new C(); // object of type C
9         A r; // obtain a reference of type A
10        r = a; // r refers to an A object
11        r.callme(); // calls A's version of callme
12        r = b; // r refers to a B object
13        r.callme(); // calls B's version of callme
14        r = c; // r refers to a C object
15        r.callme(); // calls C's version of callme
16    }
17 }
```

**Output:**

Inside A's callme method  
Inside B's callme method  
Inside C's callme method



## Why Overridden Methods?

- As stated earlier, **overridden** methods allow Java to support **run-time polymorphism**.
- **Polymorphism** is essential to object-oriented programming for one reason: it allows a **general** class to specify methods that will be common to all of its derivatives, while allowing **subclasses** to define the **specific** implementation of some or all of those methods.
- **Overridden** methods are another way that Java implements the “*one interface, multiple methods*” aspect of polymorphism.
- **Dynamic, run-time polymorphism** is one of the most powerful mechanisms that object-oriented design brings to bear on code **reuse** and **robustness**. The ability of **existing** code **libraries** to call methods on instances of **new classes** without recompiling while maintaining a clean abstract interface is a profoundly powerful tool.



# Applying Method Overriding

```
1  class Figure {
2      double dim1; double dim2;
3      Figure(double a, double b) { dim1 = a; dim2 = b; }
4      double area() {
5          System.out.println("Area for Figure is undefined.");
6          return 0;
7      }
8  }
9  class Rectangle extends Figure {
10     Rectangle(double a, double b) { super(a, b); }
11     double area() {
12         System.out.println("Inside Area for Rectangle.");
13         return dim1 * dim2;
14     }
15 }
16 class Triangle extends Figure {
17     Triangle(double a, double b) { super(a, b); }
18     double area() {
19         System.out.println("Inside Area for Triangle.");
20         return dim1 * dim2 / 2;
21     }
22 }
```

```
1  class FindAreas {
2      public static void main(String[] args) {
3          Figure f = new Figure(10, 10);
4          Rectangle r = new Rectangle(9, 5);
5          Triangle t = new Triangle(10, 8);
6          Figure figref;
7          figref = r;
8          System.out.println("Area is " + figref.area());
9          figref = t;
10         System.out.println("Area is " + figref.area());
11         figref = f;
12         System.out.println("Area is " + figref.area());
13     }
14 }
```

```
1 class FindAreas {
2     public static void main(String[] args) {
3         Figure f = new Figure(10, 10);
4         Rectangle r = new Rectangle(9, 5);
5         Triangle t = new Triangle(10, 8);
6         Figure figref;
7         figref = r;
8         System.out.println("Area is " + figref.area());
9         figref = t;
10        System.out.println("Area is " + figref.area());
11        figref = f;
12        System.out.println("Area is " + figref.area());
13    }
14 }
```

#### Output:

Inside Area for Rectangle.  
Area is 45  
Inside Area for Triangle.  
Area is 40  
Area for Figure is undefined.  
Area is 0



## Using Abstract Classes

- You can **require** that certain **methods** be **overridden** by **subclasses** by specifying the **abstract** type **modifier**. These methods are sometimes referred to as *subclasser responsibility* because they have **no implementation** specified in the **superclass**.
- Thus, a **subclass must override** them—it cannot simply use the version defined in the superclass. To declare an **abstract method**, use this general form:  
**abstract type name(parameter-list);**
- As you can see, **no method body is present**.



## Using Abstract Classes

- Any class that contains **one or more abstract methods** must also be declared **abstract**.
- To declare a **class abstract**, you simply use the **abstract** keyword in front of the class **keyword** at the beginning of the **class declaration**.
- There can be **no objects** of an **abstract class**. That is, an **abstract class** cannot be directly instantiated with the **new operator**.
- Also, you **cannot declare abstract constructors**, or **abstract static methods**.
- Any **subclass** of an **abstract class** **must** either **implement** all of the **abstract methods** in the superclass, **or** be declared **abstract itself**.



## Using Abstract Classes

- Although **abstract** classes **cannot** be used to **instantiate** objects, they can be used to **create object references**, because Java's approach to **run-time polymorphism** is implemented through the use of **superclass references**.
- Thus, it must be possible to **create a reference** to an **abstract class** so that it can be used to point to a **subclass object**.

```
1  abstract class A {
2      abstract void callme();
3      void callmetoo() {
4          System.out.println("This is a concrete method.");
5      }
6  }
7  class B extends A {
8      void callme() {
9          System.out.println("B's implementation of callme.");
10     }
11 }
12 class AbstractDemo {
13     public static void main(String[] args) {
14         B b = new B();
15         b.callme();
16         b.callmetoo();
17     }
18 }
```

```
1  abstract class Figure {
2      double dim1; double dim2;
3      Figure(double a, double b) { dim1 = a; dim2 = b; }
4      abstract double area();
5  }
6  class Rectangle extends Figure {
7      Rectangle(double a, double b) { super(a, b); }
8      double area() {
9          System.out.println("Inside Area for Rectangle.");
10         return dim1 * dim2;
11     }
12 }
13 class Triangle extends Figure {
14     Triangle(double a, double b) { super(a, b); }
15     double area() {
16         System.out.println("Inside Area for Triangle.");
17         return dim1 * dim2 / 2;
18     }
19 }
```

```
1  class AbstractAreas {
2      public static void main(String[] args) {
3          // Figure f = new Figure(10, 10); // illegal now
4          Rectangle r = new Rectangle(9, 5);
5          Triangle t = new Triangle(10, 8);
6          Figure figref; // this is OK, no object is created
7          figref = r;
8          System.out.println("Area is " + figref.area());
9          figref = t;
10         System.out.println("Area is " + figref.area());
11     }
12 }
```



## Using final with Inheritance

- The keyword **final** has **three** uses.
- **First**, it can be used to create the equivalent of a named **constant**. This use was described in the preceding chapter.
- The other **two** uses of **final** apply to **inheritance**.



## Using final to Prevent Overriding

- While **method overriding** is one of Java's most powerful features, there will be times when you will want to **prevent** it from occurring.
- To **disallow** a **method** from being **overridden**, specify **final** as a modifier at the start of its declaration.
- **Methods** declared as **final** cannot be **overridden**.
- **Methods** declared as **final** can sometimes provide a **performance enhancement**: The **compiler** is free to **inline calls** to them because it "*knows*" they will **not** be **overridden** by a **subclass**.  
**Normally**, Java **resolves** calls to methods **dynamically**, at **run time**. This is called *late binding*.  
However, since **final** methods cannot be **overridden**, a call to one can be **resolved** at **compile time**. This is called *early binding*.

```
1  class A {  
2      final void meth() {  
3          System.out.println("This is a final method.");  
4      }  
5  }  
6  class B extends A {  
7      void meth() { // ERROR! Can't override.  
8          System.out.println("Illegal!");  
9      }  
10 }
```



## Using final to Prevent Inheritance

- Sometimes you will want to **prevent** a **class** from being **inherited**.
- To do this, precede the **class declaration** with **final**.
- Declaring a **class** as **final** **implicitly** declares all of its **methods** as **final**, too.
- As you might expect, it is **illegal** to declare a class as **both abstract** and **final** since an **abstract** class is **incomplete** by itself and relies upon its **subclasses** to provide complete **implementations**.

```
1  final class A {  
2      //...  
3  }  
4  // The following class is illegal.  
5  class B extends A { // ERROR! Can't subclass A  
6      //...  
7  }
```



# Local Variable Type Inference and Inheritance

- A **superclass reference** can refer to a **derived** class **object**, and this feature is part of Java's support for **polymorphism**.
- However, it is critical to remember that, when using **local variable type inference**, the inferred type of a variable is **based** on the **declared** type of its **initializer**.
- Therefore, if the **initializer** is of the **superclass** type, that will be the **inferred** type of the **variable**.
- It does not matter if the **actual object** being referred to by the **initializer** is an **instance** of a derived class.

```
1  class MyClass {
2      // ...
3  }
4  class FirstDerivedClass extends MyClass {
5      int x;
6      // ...
7  }
8  class SecondDerivedClass extends FirstDerivedClass {
9      int y;
10     // ...
11 }
12 class TypeInferenceAndInheritance {
13     static MyClass getObj(int which) {
14         switch(which) {
15             case 0: return new MyClass();
16             case 1: return new FirstDerivedClass();
17             default: return new SecondDerivedClass();
18         }
19     }
20     public static void main(String[] args) {
21         var mc = getObj(0);
22         var mc2 = getObj(1);
23         var mc3 = getObj(2);
24     }
25 }
```



# The Object Class

- There is one **special class**, **Object**, defined by Java.
- All other **classes** are **subclasses** of **Object**.
- That is, **Object** is a **superclass** of **all other classes**.
- This means that a **reference** variable of type **Object** can **refer** to an **object** of **any** other **class**.
- Also, since **arrays** are implemented as **classes**, a **variable** of type **Object** can also **refer** to any **array**.

| Method                                                                                                                              | Purpose                                                            |
|-------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------|
| <code>Object clone( )</code>                                                                                                        | Creates a new object that is the same as the object being cloned.  |
| <code>boolean equals(Object object)</code>                                                                                          | Determines whether one object is equal to another.                 |
| <code>void finalize( )</code>                                                                                                       | Called before an unused object is recycled. (Deprecated by JDK 9.) |
| <code>Class&lt;?&gt; getClass( )</code>                                                                                             | Obtains the class of an object at run time.                        |
| <code>int hashCode( )</code>                                                                                                        | Returns the hash code associated with the invoking object.         |
| <code>void notify( )</code>                                                                                                         | Resumes execution of a thread waiting on the invoking object.      |
| <code>void notifyAll( )</code>                                                                                                      | Resumes execution of all threads waiting on the invoking object.   |
| <code>String toString( )</code>                                                                                                     | Returns a string that describes the object.                        |
| <code>void wait( )</code><br><code>void wait(long milliseconds)</code><br><code>void wait(long milliseconds,int nanoseconds)</code> | Waits on another thread of execution.                              |